Mini-projet : Radar à ultrasons piloté par réseau

Nov 6, 2026

1. Contexte et objectifs

Vous allez construire un petit « radar » : un capteur à ultrasons monté sur un servomoteur, piloté par un microcontrôleur que l'on commande à distance depuis un PC, à travers un routeur.

La difficulté du projet est réseau autant qu'embarquée : la carte est branchée côté LAN du routeur, alors que le PC est côté WAN. Le PC ne peut donc pas joindre la carte directement ; c'est à vous de configurer le routeur pour la rendre accessible depuis l'extérieur.

Ce module de 4 séances de 4 h, suivies d'une 5e séance de restitution devant un jury, réunit ce que vous avez vu en langage C, en microcontrôleurs et en réseau.

Objectifs pédagogiques

À la fin du projet, vous saurez :

- piloter un servomoteur (signal PWM) et mesurer une distance avec un capteur à ultrasons (mesure d'une durée d'impulsion) depuis un microcontrôleur STM32, en C, avec Keil µVision et le Middleware Ethernet ;
- écrire sur la carte un serveur UDP qui interprète des commandes textuelles ;
- écrire en C, sur PC, une application graphique (Qt) qui joue le rôle de client réseau avec l'API sockets (UDP) ;
- configurer un routeur : réservation DHCP statique et redirection de port (NAT) ;
- diagnostiquer une panne réseau avec les outils usuels (ping, netcat, Wireshark) ;
- tester, intégrer et documenter un système complet, en équipe.

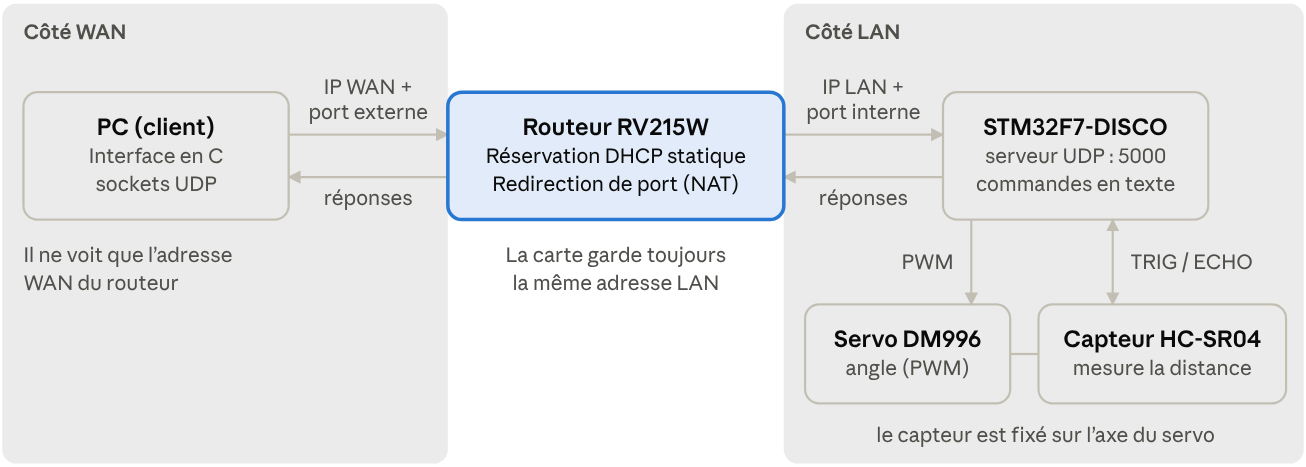
Résultat attendu

En démonstration finale, depuis le PC, vous envoyez une commande pour orienter le capteur à un angle donné, puis une commande de balayage (par exemple de 0° à 180° par pas de 10°). Votre application affiche les distances mesurées pour chaque angle.

2. Architecture du système

Le PC (côté WAN) n'atteint la carte (côté LAN) que par l'adresse WAN du routeur : c'est la redirection de port qui transmet chaque requête à la carte, dont l'adresse LAN est fixée par la réservation DHCP.

Le PC n'atteint la carte que par la redirection de port du routeur



architecture · PC côté WAN, routeur, carte côté LAN

Le PC envoie ses commandes à l'adresse WAN du routeur, sur un port externe. Le routeur les redirige vers l'adresse LAN de la carte et le port du service UDP, puis renvoie les réponses au PC. Choisissez un port externe différent du port interne (par exemple 8000 à l'extérieur pour 5000 à l'intérieur) : vous vérifierez ainsi que la traduction de port fonctionne vraiment.

3. Matériel et outils

Chaque binôme ou groupe dispose d'un poste de travail identique. Le projet template pour Keil µVision se trouve sur le lecteur réseau, dans le dossier \\COMMUN\CORCOLLE\Mini-Projet BUT3ESE LPMECSE. Copiez-le sur votre poste avant de l'ouvrir.

- une carte STM32F7-DISCO, programmée avec Keil µVision, la librairie graphique emWin, et le Middleware Ethernet : vous partez d'un projet template fourni, dans lequel vous modifiez l'adresse MAC pour la rendre unique ; l'adresse IP obtenue par DHCP s'affiche sur l'écran de la carte ;
- un servomoteur DM996 et un capteur à ultrasons HC-SR04, fixés l'un à l'autre ;
- un routeur Cisco RV215W, dont vous avez les droits d'administration via son interface web, ouverte dans le navigateur de la Raspberry Pi ;
- un PC côté WAN avec un compilateur C, Qt et son environnement de développement, Wireshark et netcat ;
- une Raspberry Pi côté LAN, avec son écran, son bureau et son navigateur, qui sert à configurer le routeur et à tester la carte sans passer par la redirection de port ;
- une plaque d'essais, des câbles et une alimentation adaptée au servo.

Composant	Liaison à la carte	Point d'attention
Servomoteur DM996	Signal PWM sur PF8, alimentation, masse	Vérifiez dans sa documentation la plage d'impulsion acceptée
Capteur HC-SR04	TRIG sur PB14 (sortie), ECHO sur PG6 (entrée), 5 V, masse	ECHO sort en 5 V : la broche PG6 accepte le 5 V, donc le branchement est direct, sans adaptation de niveau

L'utilisation de GitHub est recommandée : un dépôt partagé par le binôme permet de versionner le projet Keil µVision et l'application Qt, et de travailler chacun dans sa copie avant de fusionner. Vous donnerez l'accès à ce dépôt aux enseignants, et vous y enverrez vos modifications régulièrement pendant les séances : son historique est consulté lors de l'évaluation.

4. Spécifications fonctionnelles

La carte propose un service UDP sur le port 5000, qui comprend un langage de commandes textuelles.

Commandes

Commande	Effet	Réponse
`ANGLE <a>`	Oriente le servo à a degrés (0 à 180)	`OK` ou `ERR <motif>`
`DIST`	Mesure la distance à l'angle courant	`DIST <cm>`, ou `DIST TIMEOUT` si aucun écho
`SCAN <début> <fin> <pas>`	Balaye de début à fin et mesure à chaque angle	un seul datagramme : une ligne `<angle> <cm>` par mesure, puis `END`

Règles du protocole

- Les messages sont en ASCII et se terminent par un saut de ligne (`\n`). Les distances sont en centimètres, avec une décimale.
- En UDP, un datagramme contient une commande, et la réponse revient dans un datagramme vers l'adresse et le port de l'expéditeur.
- UDP ne garantit ni la livraison ni l'ordre : un datagramme peut être perdu. Le client doit prévoir un délai d'attente et un nouvel essai. La réponse à SCAN doit tenir dans un seul datagramme : limitez le nombre de mesures.
- Une commande inconnue, mal formée ou hors plage (angle > 180, par exemple) reçoit une réponse `ERR` ; la carte ne doit jamais se bloquer.

Application PC

L'application PC, dotée d'une interface graphique Qt et d'un module réseau écrit en C, doit :

- permettre de saisir l'adresse du routeur et le port dans l'interface, sans les coder en dur ;
- commander l'angle du servo (`ANGLE`) et déclencher une mesure (`DIST`), afficher la réponse, et gérer l'absence de réponse (délai d'attente puis nouvel essai) ;

- lancer un balayage (`SCAN`) depuis l'interface, lire la réponse, séparer les lignes jusqu'à `END` et afficher le tableau angle / distance ;
- rester réactive pendant l'attente d'une réponse : l'interface ne doit pas se figer ;
- fermer proprement ses sockets et afficher les erreurs à l'utilisateur dans l'interface.

Rappels techniques

- Servomoteur : signal périodique de 50 Hz ; la largeur d'impulsion, typiquement de 1 ms à 2 ms, fixe l'angle. Calibrez les bornes sur votre servo.
- Capteur HC-SR04 : une impulsion de 10 µs sur TRIG déclenche la mesure ; la durée de l'impulsion sur ECHO correspond à l'aller-retour du son, soit une distance en cm d'environ la durée en µs divisée par 58.
- Laissez le servo se stabiliser après chaque déplacement avant de mesurer, et espacez deux mesures d'au moins 60 ms.
- Sans écho au bout d'environ 30 ms, abandonnez la mesure et répondez `TIMEOUT`.
- Sur la STM32F7, utilisez un timer en mode PWM pour le servo (PF8), et un timer pour mesurer la durée de l'impulsion sur ECHO (PG6).

5. Tâches à réaliser

Le projet suit quatre séances de 4 h. Chaque séance se termine par un jalon que l'enseignant valide avant que vous passiez à la suite ; cochez les tâches au fur et à mesure.

Vous travaillez en binôme et vous répartissez les tâches. En séances 1 et 2, un étudiant A s'occupe du réseau et du routeur, un étudiant B de la carte et de ses périphériques ; vous partagez la carte : chacun développe dans sa copie du projet et la programme à tour de rôle. À partir de la séance 3, vous pouvez vous séparer : l'un termine la partie microcontrôleur, l'autre développe l'application Qt sur le PC. Vous vous retrouvez pour l'intégration, et vous devez tous les deux pouvoir tout expliquer lors de la démonstration.

Séance 1 – Le matériel en local

Objectif : prendre en main la carte avec quelques périphériques simples, puis faire fonctionner le servo sans réseau.

Ensemble

- [ ] Retrouver Keil µVision (déjà utilisé en module microcontrôleur) : copier le projet template depuis le lecteur réseau (voir section 3), l'ouvrir, compiler, programmer la carte et la déboguer.
- [ ] Créer un dépôt GitHub pour le binôme (recommandé), y déposer le projet template et en donner l'accès aux enseignants.

Étudiant A – réseau et routeur

- [ ] Repérer le routeur RV215W et s'y connecter depuis la Raspberry Pi : adresse de son interface web, identifiants, plan d'adressage LAN et WAN de votre poste.
- [ ] Dans le projet template, donner à la carte une adresse MAC unique (dans le fichier Net_Config_ETH_0.h), la brancher au LAN du routeur et relever l'adresse IP obtenue par DHCP, affichée sur l'écran.
- [ ] Configurer sur le routeur une réservation DHCP statique (menu Networking > LAN > Static DHCP du RV215W) : cette MAC reçoit toujours la même adresse IP. Vérifier après un redémarrage de la carte.

Étudiant B – carte et périphériques

- [ ] Échauffement 1 : un interrupteur (switch) de l'interface graphique de la carte allume et éteint une LED.
- [ ] Échauffement 2 : un curseur (slider) règle la tension de sortie du DAC.
- [ ] Échauffement 3 : une mollette (rotary) règle le rapport cyclique d'un signal PWM.
- [ ] Échauffement 4 : une jauge (gauge) affiche la position du potentiomètre, lue par le convertisseur analogique-numérique (ADC).
- [ ] Écrire `servo_set_angle(int deg)` : signal PWM à 50 Hz, bornes d'impulsion calibrées sur votre servo.
- [ ] Écrire une boucle de test qui balaie le servo de 0° à 180° et affiche l'angle sur l'écran de la carte.

Pour les exercices d'échauffement, le projet template contient le fichier `ID_SCREEN_00_Slots.c`, généré par AppWizard (SEGGER). Il reçoit les actions de l'utilisateur sur l'écran de la carte. Vous n'y écrivez que dans les fonctions, à l'endroit signalé par le commentaire « code à exécuter à mettre ici » ; le fichier déclare déjà `stm32f7xx.h` pour accéder aux registres.

Élément de l'écran	Fonction à compléter	Valeur disponible
Switch	`ID_SCREEN_00__ID_SWITCH_00__WM_NOTIFICATION_VALUE_CHANGED`	`Val` vaut 0 ou 1
Slider	`ID_SCREEN_00__ID_SLIDER_00__WM_NOTIFICATION_VALUE_CHANGED`	`Val` entre 0 et 100 : à convertir en consigne du DAC
Rotary	`ID_SCREEN_00__ID_ROTARY_00__WM_NOTIFICATION_VALUE_CHANGED`	`Val` entre -900 et 900 : à convertir en rapport cyclique
Gauge	`ID_SCREEN_00__ID_TIMER_00__APPW_NOTIFICATION_TIMER`	Appelée périodiquement par le timer `ID_TIMER_00` : lisez-y le potentiomètre pour mettre à jour la jauge

Dans les trois fonctions d'un widget, la valeur se lit avec `APPW_GetValue(ID_SCREEN_00, <identifiant du widget>, NULL)`.

L'initialisation des périphériques (GPIO de la LED, DAC, PWM, ADC, puis plus tard le servo et le capteur) s'écrit dans la fonction `main`, entre les commentaires « Début Initialisation périphérique » et « Fin Initialisation périphérique », à l'endroit marqué « Initialiser vos périphériques ici ». Le `main` fourni appelle d'abord `configuration_overclock()`, nécessaire pour utiliser l'écran LCD. Avec le RTOS (Keil RTX5), il crée ensuite `Init_Ethernet()` et `Init_GUIThread()`, puis lance `osKernelStart()`. La boucle `while (1)` finale ne s'exécute pas avec le RTOS : n'y mettez aucun code. Pour vos essais en boucle (par exemple la mesure de distance ou le balayage du servo), le template fournit une tâche de test vide, Thread_Test(), créée par Init_TestThread() juste avant osKernelStart() : vous pouvez écrire votre boucle de test dans cette tâche, qui se répète toutes les 100 ms grâce à osDelay(100).

Jalon : le servo balaie 0° à 180° et l'angle affiché correspond à sa position réelle ; la carte obtient toujours la même adresse IP, affichée sur son écran.

Séance 2 – Routeur et commandes UDP

Objectif : mettre en œuvre le capteur, puis commander le servo et mesurer par UDP depuis le côté WAN.

Étudiant B – capteur

- [ ] Écrire `distance_cm(void)` : impulsion TRIG, mesure de la durée de ECHO, conversion en cm, délai maximal.
- [ ] Écrire une boucle de test qui balaie le servo et affiche angle et distance sur l'écran de la carte.

Étudiant A – UDP et routeur

- [ ] Écrire le serveur UDP de la carte (port 5000), avec l'API UDP du Middleware (voir `UDP_Socket.c` ci-dessous) : lecture d'un datagramme, analyse de `ANGLE` et `DIST`, réponse à l'expéditeur.
- [ ] Configurer sur le routeur la redirection (Single Port Forwarding) d'un port externe UDP vers la carte, port 5000.

Pour le serveur UDP, le projet template contient `UDP_Socket.c`, un modèle de code du Middleware Network (Arm MDK). Il est à compléter et à adapter :

Fonction	Rôle
`Init_Ethernet()`	Initialise la pile réseau avec `netInitialize`. La création du socket (`netUDP_GetSocket`) et l'ouverture du port (`netUDP_Open`) sont en commentaire : à activer, avec le port 5000 à la place du port 2000 de l'exemple

Fonction	Rôle
`udp_cb_func()`	Fonction de rappel appelée à chaque datagramme reçu. Elle reçoit l'adresse de l'expéditeur (`addr`), les données (`buf`) et leur longueur (`len`) : c'est ici que vous analysez la commande
`send_udp_data()`	Exemple d'envoi : `netUDP_GetBuffer` réserve un tampon, vous le remplissez, puis `netUDP_Send` l'envoie à une adresse `NET_ADDR`

Pour répondre au PC, réutilisez l'adresse `addr` reçue par la fonction de rappel. Ne supposez pas que `buf` se termine par un `\0` : appuyez-vous sur `len` .

Ensemble

- [ ] Brancher le servo et le capteur sur le serveur UDP, puis tester depuis le PC avec netcat (`nc -u`) en visant l'adresse WAN du routeur, et observer l'échange dans Wireshark.

netcat (`nc`) est un outil en ligne de commande qui envoie et reçoit des données brutes sur le réseau, en TCP ou en UDP. Il vous sert de client de test avant d'avoir votre propre application : par exemple, `echo "ANGLE 45" | nc -u <adresse WAN du routeur> <port externe>` envoie la commande en UDP. Si le servo bouge, la chaîne PC, routeur et carte fonctionne. Si netcat n'est pas installé sur votre PC, utilisez `ncat` , fourni avec Nmap. Vous pouvez aussi tester depuis la Raspberry Pi branchée côté LAN : vous visez alors directement l'adresse IP de la carte et son port 5000, sans passer par la redirection. Cela permet de valider le serveur UDP avant de configurer le routeur.

Où écrire une boucle de test ? Avec le RTOS, la boucle `while (1)` à la fin de `main()` ne s'exécute jamais : `osKernelStart()` ne rend pas la main. Le projet template contient donc une tâche de test vide, `Thread_Test()` , créée dans `main()` et répétée toutes les 100 ms grâce à `osDelay(100)` . Placez-y vos essais sur le capteur et le servo (mesure en boucle, balayage de l'angle) sans attendre de commande réseau. N'écrivez surtout pas de boucle dans `udp_cb_func()` , qui bloquerait la réception des datagrammes, ni dans la tâche de l'interface graphique, qui figerait l'écran. Une fois le serveur UDP en place, désactivez cette tâche (ou videz sa boucle) pour qu'elle n'entre pas en conflit avec les commandes reçues du PC.

Jalon : la distance affichée coïncide avec une mesure à la règle ; depuis le PC, une commande `ANGLE 45` envoyée avec netcat fait bouger le servo, et `DIST` renvoie une mesure.

Séance 3 – Application PC graphique et balayage

Objectif : remplacer netcat par votre propre application graphique, fiable malgré les pertes UDP.

Étudiant A – application PC

- [ ] Écrire le client UDP en C (`socket` , `sendto` , `recvfrom`) avec délai d'attente et nouvel essai, et le tester d'abord sans interface.
- [ ] Dans le client C, envoyer `SCAN` , lire la réponse et séparer les lignes jusqu'à `END` .
- [ ] Réaliser une première version de l'interface graphique avec Qt : champs d'adresse et de port, commande d'angle, boutons de mesure et de balayage, zone d'affichage du tableau angle / distance.
- [ ] Garder l'interface réactive pendant l'attente d'une réponse : le réseau ne doit pas figer la fenêtre.

Étudiant B – carte

- [ ] Ajouter `SCAN` au serveur de la carte : balayage de début à fin, réponse en un seul datagramme terminée par `END` .
- [ ] Rendre le serveur robuste : réponse `ERR` à toute commande inconnue, mal formée ou hors plage, sans jamais bloquer la carte.

Ensemble

- [ ] Tester l'application PC avec la carte, à travers le routeur, et corriger les écarts de protocole.

Jalon : votre application graphique pilote le servo en UDP et récupère un balayage complet, à travers le routeur.

Séance 4 – Intégration, robustesse et démonstration

Objectif : fiabiliser, documenter et préparer la restitution.

- [ ] Tester les cas limites : commande inconnue, angle hors plage, datagramme perdu, redémarrage de la carte.
- [ ] Finaliser l'interface Qt : mise en forme, validation des saisies (adresse, port, angle), messages d'erreur visibles, fermeture propre.
- [ ] Rédiger le compte rendu (voir section 6), avec captures de la configuration du routeur et d'une capture Wireshark.
- [ ] Préparer la restitution de la séance 5.

Jalon : démonstration complète d'un poste neuf (carte et routeur remis à zéro) jusqu'au balayage affiché sur le PC.

Séance 5 – Restitution devant un jury

Objectif : présenter votre réalisation, en binôme, à un jury de deux enseignants qui vous posera des questions.

- [ ] Présenter votre système et en faire la démonstration complète, depuis l'interface graphique du PC jusqu'au servo et au capteur.
- [ ] Répondre aux questions du jury sur les deux parties du projet, la carte et le réseau avec l'application PC, quelle que soit la partie que vous avez réalisée.

Le format du passage (durée, ordre) est communiqué par les enseignants.

6. Livrables et évaluation

Vous êtes évalués sur un système qui fonctionne de bout en bout, sur la qualité de votre code et sur votre compréhension du réseau.

Livrables

- le projet Keil µVision de la carte, commenté et compilable ;
- le code source de l'application graphique du PC;
- l'accès de vos enseignants à votre dépôt GitHub (lien, ou invitation si le dépôt est privé), avec des envois réguliers tout au long du projet ;
- un compte rendu court contenant : le schéma de câblage, le plan d'adressage, la configuration du routeur (réservation DHCP et redirections, avec captures d'écran), la description du protocole, une capture Wireshark commentée et les difficultés rencontrées ;
- la restitution devant le jury en séance 5 : démonstration, puis questions.

Critères d'évaluation

Critère	Ce qui est regardé
Fonctionnement	La démonstration réalise `ANGLE`, `DIST` et `SCAN` depuis l'interface graphique du PC, à travers le routeur
Qualité du code C	Fonctions courtes et nommées clairement, constantes plutôt que nombres magiques, retours d'erreur testés, sockets fermées
Robustesse	Les cas limites de la séance 4 ne bloquent ni la carte ni l'application
Réseau	Configuration correcte du routeur et capacité à expliquer ce qui se passe sur le réseau
Compte rendu	Complet, clair, reproductible par un autre groupe
Avancement et travail d'équipe	Jalons atteints à chaque séance, rôles partagés, historique GitHub régulier : des envois répartis sur toute la durée du projet sont mieux valorisés qu'un seul envoi la veille du rendu

Les extensions de la section 7 peuvent rapporter des points bonus. Le barème et la date de rendu sont communiqués par l'enseignant.

Questions à maîtriser pour la restitution

- Quelle adresse et quel port le PC vise-t-il, et quelle adresse la carte voit-elle comme source ?
- Pourquoi la redirection de port cesserait-elle de fonctionner sans réservation DHCP statique ?
- Que se passe-t-il si un datagramme UDP est perdu, et comment votre application le gère-t-elle ?
- Pourquoi un serveur UDP n'a-t-il pas de connexion à accepter, et comment connaît-il l'adresse de l'expéditeur ?
- Quelle partie avez-vous réalisée, et comment s'articule-t-elle avec celle de votre binôme ?

7. Conseils, pièges classiques et points bonus

Testez par couches : un système qui ne répond pas a toujours une cause précise, et on la trouve en validant chaque maillon l'un après l'autre.

Méthode de diagnostic

1. Le capteur et le servo fonctionnent-ils en local, sans réseau ?
2. La carte répond-elle à netcat depuis un poste placé côté LAN (la Raspberry Pi, par exemple) ?
3. La carte a-t-elle bien l'adresse IP attendue par la réservation DHCP (celle affichée sur son écran) ?
4. La règle de redirection porte-t-elle sur le bon protocole (UDP), le bon port externe, la bonne adresse et le bon port interne ?
5. Le PC vise-t-il l'adresse WAN du routeur, et non l'adresse LAN de la carte ?
6. Que montre Wireshark : les paquets partent-ils, arrivent-ils, une réponse revient-elle ?

Pièges fréquents

- Une règle de redirection déclarée en TCP ne laisse pas passer l'UDP : vérifiez le protocole de la règle.

- netcat ou un système Windows peut ajouter `\r` avant `\n` : acceptez les deux fins de ligne côté carte.
- Pensez à terminer vos chaînes par un `\0` avant de les analyser, et ne dépassez jamais la taille de vos tampons.
- Une mesure ne doit pas bloquer la carte indéfiniment : gardez le délai maximal sur ECHO.
- Si la carte redémarre quand le servo bouge, c'est presque toujours un problème d'alimentation.
- Gardez une adresse MAC unique : deux cartes avec la même adresse MAC sur un réseau perturbent la réservation DHCP.
- Le pare-feu du PC peut bloquer les réponses entrantes.

Points bonus

Si vous avez terminé en avance, ces extensions sont prises en compte dans l'évaluation sous forme de points bonus :

- filtrer les mesures (moyenne ou médiane de plusieurs mesures par angle) ;
- afficher le balayage sous forme de radar en demi-cercle dans l'interface, et l'exporter en CSV ;
- permettre de régler le pas du balayage depuis l'interface Qt ;
- ajouter un numéro de séquence aux messages pour détecter les pertes ;
- ajouter un mot de passe (un « jeton ») au début de chaque commande : la carte le vérifie et ignore toute commande qui ne le contient pas.